Module 01 — The Filesystem & File Operations

1.1 The core philosophy: "everything is a file"

In Linux, almost everything is represented as a file: your documents, yes, but also directories, hard drives, USB devices, network sockets, running processes (`/proc`), and even hardware information (`/sys`). This isn't just trivia — it's why a small set of tools (`cat` , `ls` , `chmod` , `find` , redirection) gives you so much power. Once you can manipulate "files," you can manipulate almost the entire system.

There are no drive letters (`C:` , `D:`) in Linux. There is **one single tree**, starting at `/` (called "root", not to be confused with the root *user*). Everything — including other physical disks, USB drives, and network shares — gets *mounted* somewhere into that one tree.

1.2 The Filesystem Hierarchy Standard (FHS)

Run `ls -la /` right now and you'll see the top-level directories. Here's what each one is for, and why you'll care as a DevOps engineer:

Directory	Purpose	DevOps relevance
`/bin` , `/usr/bin`	Essential user command binaries	On modern Ubuntu, `/bin` is a symlink to `/usr/bin`
`/sbin` , `/usr/sbin`	System administration binaries (often need root)	`iptables` , `useradd` , etc. live here
`/etc`	System-wide **configuration files**	You will live here. SSH config, cron, systemd units, app configs
`/var`	**Variable** data — changes while the system runs	`/var/log` (logs!), `/var/lib` (app data), `/var/www` (web content)
`/home`	Personal directories for regular users	`/home/<you>`
`/root`	Home directory for the `root` user	Only root can access this
`/usr`	User-installed (but OS-managed) programs & data	The bulk of the installed system lives here
`/usr/local`	Software installed manually/outside package manager	Custom scripts often go in `/usr/local/bin`
`/opt`	Optional/third-party software packages	Many vendor tools (e.g., some monitoring agents) install here
`/tmp`	Temporary files, **cleared on reboot**	Scratch space — never store anything important here
`/proc`	Virtual filesystem exposing **kernel & process info**	Doesn't exist on disk — generated live by the kernel

Directory	Purpose	DevOps relevance
`/sys`	Virtual filesystem exposing **kernel/hardware/device info**	Used to tune kernel parameters at runtime
`/dev`	Device files (disks, terminals, null device, etc.)	`/dev/sda` , `/dev/null` , `/dev/tty`
`/boot`	Bootloader files, kernel images	Mostly irrelevant inside WSL
`/mnt`	Temporary/manual mount points	In WSL, `/mnt/c` , `/mnt/d` are your **Windows drives**
`/media`	Auto-mounted removable media (USB, CDs)	
`/srv`	Data for services hosted on this system	Sometimes used for web/git server data
`/lib` , `/usr/lib`	Shared libraries needed by binaries in `/bin` , `/sbin`	

■ **DevOps relevance**: When something breaks, you'll instinctively know *where to look*. App crashed? `/var/log/<app>` . Config wrong? `/etc/<app>` . Disk full? Probably `/var/log` or `/tmp` . This map becomes second nature fast.

1.3 Paths: absolute vs relative

An **absolute path** always starts with `/` and describes the full route from the root of the tree:

```
/home/alex/projects/app/config.yaml
```

A **relative path** is relative to your **current working directory**:

```
projects/app/config.yaml      # relative — assumes you're in /home/alex
../config.yaml                # ".." means "go up one directory"
./run.sh                       # "." means "this directory" — explicit form is often required
```

Special path shortcuts:

Symbol	Meaning
`.`	Current directory
`..`	Parent directory
`~`	Your home directory (`/home/<you>`)
`~username`	Another user's home directory
`-`	(with `cd` only) the *previous* directory you were in

1.4 Navigating the filesystem

```
pwd                 # Print Working Directory — "where am I?"
cd /etc             # change directory (absolute path)
```

```
cd ssh              # change directory (relative — now in /etc/ssh)
cd ..               # go up one level (back to /etc)
cd                  # no argument = go home (~)
cd ~                # also home
cd -                # toggle back to the previous directory
```

`ls` — listing directory contents

```
ls              # simple list
ls -l           # "long" format — permissions, owner, size, date
ls -a           # show hidden files (dotfiles) too
ls -la          # combine: long format + hidden files
ls -lh          # human-readable sizes (K, M, G instead of bytes)
ls -lt          # sort by modification time, newest first
ls -lS          # sort by size, largest first
ls -lR          # recurse into subdirectories
ls -d */        # list only directories in the current location
```

Reading `ls -l` output

```
$ ls -l /etc/hostname
-rw-r--r-- 1 root root 12 Jun 10 09:15 /etc/hostname
```

Breaking this down field by field:

```
-rw-r--r--   1   root   root   12   Jun 10 09:15   /etc/hostname
■■■■■■■■■■   ■   ■■■■■  ■■■■■  ■    ■■■■■■■■■■■■    ■■■■■■■■■■■■■■■
■ ■  ■  ■    ■     ■      ■    ■         ■               ■
■ ■  ■  ■    ■     ■      ■    ■         ■               ■■ filename
■ ■  ■  ■    ■     ■      ■    ■         ■■ last modified time
■ ■  ■  ■    ■     ■      ■    ■■ size in bytes
■ ■  ■  ■    ■     ■      ■■ group owner
■ ■  ■  ■    ■     ■■ user owner
■ ■  ■  ■    ■■ number of hard links
■ ■  ■  ■■ "other" permissions (everyone else)
■ ■  ■■ "group" permissions
■ ■■ "owner/user" permissions
■■ file type (- = regular file)
```

We'll dissect the permission bits (`rw-r--r--`) thoroughly in section 1.8.

1.5 File types

That very first character in `ls -l` tells you the **type** of entry:

Symbol	Type	Example
`-`	Regular file	a text file, binary, script
`d`	Directory	
`l`	Symbolic link (symlink)	`/bin -> usr/bin`
`c`	Character device	`/dev/tty` , `/dev/null`
`b`	Block device	`/dev/sda`
`s`	Socket	used for inter-process communication

Symbol	Type	Example
p	Named pipe (FIFO)	

You can also use the `file` command to identify what something actually is, regardless of its extension:

```
file /etc/hostname
file /bin/ls
file ~/playground
```

1.6 Creating, copying, moving, and deleting

```
cd ~/playground

# Create files
touch notes.txt              # creates an empty file, or updates timestamp if it exists
touch a.txt b.txt c.txt       # create multiple at once

# Create directories
mkdir logs
mkdir -p projects/app/src     # -p creates all parent directories as needed, no error if exists

# Copy
cp notes.txt notes-backup.txt
cp -r projects projects-copy   # -r = recursive, REQUIRED for directories
cp -p notes.txt notes2.txt      # -p preserves permissions/timestamps
cp -v notes.txt notes3.txt      # -v = verbose (prints what it's doing)
cp -i notes.txt notes.txt       # -i = interactive (asks before overwrite)

# Move / rename (same command does both — Linux doesn't distinguish)
mv notes3.txt renamed.txt
mv renamed.txt logs/

# Remove
rm a.txt
rm -i b.txt          # asks for confirmation
rm -r projects-copy  # recursive, for directories
rm -rf projects-copy # recursive + force (no prompts, ignores nonexistent files)

rmdir logs            # only works on EMPTY directories
```

■■ ■ `rm -rf` **is the most dangerous command you'll learn today.** It deletes recursively, forcefully, with no confirmation, and **no undo, no recycle bin**. `rm -rf /` would (try to) delete your entire system — modern tools block this specific case, but `rm -rf ./*` or a mistyped path with a stray space (`rm -rf ~ /oldfiles` instead of `rm -rf ~/oldfiles`) absolutely will not stop you. **Always double-check the path before pressing Enter**, especially with wildcards. Many experienced engineers `cd` into the target directory, run `ls` to confirm what's there, *then* run `rm -rf ./*` .

1.7 Wildcards (globbing)

The shell expands these patterns into matching filenames *before* the command even runs:

```
ls *.txt          # all files ending in .txt
ls a*             # all files starting with "a"
ls ?.txt          # ? matches exactly ONE character: a.txt, b.txt, but not ab.txt
ls [ab].txt       # matches a.txt or b.txt (character set)
ls [a-c].txt      # matches a.txt, b.txt, c.txt (range)
ls file{1,2,3}.txt  # brace expansion: file1.txt file2.txt file3.txt (not technically "globbing" but related)
```

■ **DevOps relevance**: `rm -f /var/log/myapp/*.log.gz` (clean up compressed logs), `cp configs/*.yaml /etc/myapp/` — wildcards are everywhere in scripts.

1.8 Viewing file contents

```
cat notes.txt          # dump entire file to terminal
cat -n notes.txt        # with line numbers
tac notes.txt           # like cat, but reversed (last line first)

less /var/log/syslog    # paginated viewer — THE tool for big files
# Inside less:  space = next page, b = back page, /pattern = search,
#               n = next match, q = quit, g = top, G = bottom

head notes.txt          # first 10 lines
head -n 20 notes.txt     # first 20 lines
tail notes.txt           # last 10 lines
tail -n 50 notes.txt      # last 50 lines
tail -f /var/log/syslog    # FOLLOW the file — show new lines as they're written
```

■ `tail -f` **is one of the most-used DevOps commands of all time.** Deploying an app and want to watch its logs live as requests come in? `tail -f /var/log/myapp/access.log` . It's the simplest possible real-time monitoring.

Use `head` and `tail` together to peek at the middle of huge files, or `wc -l file` to count total lines first.

1.9 Links: hard links vs symbolic links

Every file is actually stored as data blocks on disk, referenced by an **inode** (a data structure holding metadata: permissions, owner, size, and pointers to the data blocks — but *not* the filename). A directory entry is really just a mapping from a **name** to an **inode number**.

```
ls -i notes.txt    # shows the inode number
```

Hard links

A hard link is a *second name pointing to the same inode*. There's no "original" — they're equally real, and the data is only freed once **all** hard link names referencing it are deleted.

```
echo "hello" > original.txt
ln original.txt hardlink.txt
ls -li original.txt hardlink.txt   # same inode number, link count = 2
echo "more text" >> hardlink.txt
cat original.txt                    # shows "more text" too — same data!
```

Limitations: hard links can't cross filesystems (e.g., can't link from `/home` to `/mnt/c` if they're different filesystems) and can't point to directories.

Symbolic links (symlinks)

A symlink is a tiny special file that just *contains a path* to another file. It's a "shortcut."

```
ln -s /etc/hostname hostname-link
ls -l hostname-link
# lrwxrwxrwx 1 you you 12 ... hostname-link -> /etc/hostname
cat hostname-link    # transparently reads /etc/hostname
```

If you delete the target, the symlink becomes "broken" (dangling) but still exists. Symlinks *can* cross filesystems and point to directories — they're used constantly for things like "current version" pointers (`/usr/bin/python3 -> python3.12`).

	Hard link	Symbolic link
Points to	Inode (data) directly	A path (string)
Cross filesystem	No	Yes
Can link directories	No	Yes
If original deleted	Data still accessible via hard link	Symlink becomes broken
Created with	`ln target linkname`	`ln -s target linkname`

1.10 Permissions — the foundation of Linux security

Every file and directory has an **owner** (a user) and a **group**, plus three sets of permissions: for the **owner**, for the **group**, and for **everyone else** ("other").

```
-rw-r--r--
 ■■■ ■■■ ■■■
 ■■■ ■■■ ■■■■ "other" permissions
 ■■■ ■■■■■■■■ "group" permissions
 ■■■■■■■■■■■■ "owner" permissions
```

Each set has three possible flags:

Flag	On a file	On a directory
`r` (read)	Can view file contents	Can list directory contents (`ls`)
`w` (write)	Can modify/delete file contents	Can create/delete/rename files *inside* it
`x` (execute)	Can run it as a program/script	Can `cd` into it / access files inside (even if you can't list them)

■ A directory with `r-x` permissions lets you access files inside *if you know their exact name*, but you can't `ls` it. A directory with `--x` only is the basis of "you can go through, but can't see what's there."

Viewing and changing ownership

```
ls -l notes.txt
chown alex notes.txt           # change owner to "alex"
chown alex:developers notes.txt # change owner AND group in one go
chgrp developers notes.txt      # change just the group
chown -R alex:developers projects/  # recursive — for whole directory trees
```

`chmod` — changing permissions

There are two notations: **symbolic** and **octal (numeric)**.

Symbolic notation:

```
chmod u+x script.sh     # add execute for the owner (u = user/owner)
chmod g-w notes.txt      # remove write from group
```

```
chmod o=r notes.txt       # set "other" to exactly read-only
chmod a+r notes.txt        # a = all (user, group, other)
chmod u+rwx,g+rx,o-rwx script.sh  # combine multiple, comma-separated
```

`u` = user/owner, `g` = group, `o` = other, `a` = all. `+` adds, `-` removes, `=` sets exactly.

Octal notation — each permission set is a 3-bit number, where `r=4, w=2, x=1`, summed:

Octal	Binary	rwx	Meaning
7	111	rwx	read + write + execute
6	110	rw-	read + write
5	101	r-x	read + execute
4	100	r--	read only
0	000	---	nothing

So a permission string is three digits: owner, group, other.

```
chmod 755 script.sh   # rwxr-xr-x : owner can do everything, everyone else can read+execute
chmod 644 notes.txt    # rw-r--r-- : owner read/write, everyone else read-only — the typical "config file" permi
chmod 600 id_rsa        # rw------- : owner only — typical for SSH private keys
chmod 700 ~/.ssh          # rwx------ : owner-only directory access
chmod -R 644 configs/      # apply recursively
```

■ **DevOps relevance**: SSH will *refuse to use* a private key if its permissions are too open (e.g., `644`). `chmod 600 ~/.ssh/id_ed25519` is a rite of passage. Similarly, web servers run as low-privilege users (`www-data`), so deployed files need correct ownership/permissions or the app can't read them — "permission denied" errors are one of the most common things you'll debug.

`umask` — default permissions

When you create a new file or directory, the OS doesn't give it full `666` / `777` — it subtracts the `umask`.

```
umask          # show current umask, typically 0022
```

- New files start at `666` (rw-rw-rw-), then umask `022` is subtracted → `644` (rw-r--r--)
- New directories start at `777`, minus `022` → `755` (rwxr-xr-x)

You can set it (usually in `~/.bashrc` or `/etc/profile`) for the session: `umask 027` would make new files more restrictive (no group write, no access for others).

1.11 Special permission bits: SUID, SGID, sticky bit

These are the 4th, less-commonly-seen permission concepts — represented as a leading 4th digit in octal (`4755`, `2755`, `1777`), or as special characters in `ls -l`.

Bit	Octal	Symbolic effect	Meaning
SUID (Set User ID)	`4000`	`s` instead of `x` in owner's execute position	When run, the program executes with the **file owner's** privileges, not the caller's

Bit	Octal	Symbolic effect	Meaning
SGID (Set Group ID)	`2000`	`s` instead of `x` in group's execute position	On a file: runs with the file's group privileges. On a *directory*: new files inside inherit the directory's group
Sticky bit	`1000`	`t` instead of `x` in other's execute position	On a directory: only the file's owner (or root) can delete/rename files inside it, even if others have write access

Classic real examples:

```
ls -l /usr/bin/passwd
# -rwsr-xr-x ... /usr/bin/passwd
```

The `s` means: any user can run `passwd`, and while it runs, it temporarily has root privileges — necessary because changing your password means writing to `/etc/shadow`, which normal users can't touch directly.

```
ls -ld /tmp
# drwxrwxrwt ... /tmp
```

The `t` (sticky bit) means: everyone can create files in `/tmp` (that `rwxrwxrwx`), but you can only delete *your own* files there, not other users'.

Setting them:

```
chmod 4755 myprogram     # SUID + rwxr-xr-x
chmod 2775 shared_dir      # SGID + rwxrwxr-x
chmod 1777 shared_tmp       # sticky bit + rwxrwxrwx
chmod u+s myprogram           # symbolic SUID
chmod g+s shared_dir            # symbolic SGID
chmod +t shared_tmp               # symbolic sticky bit
```

■■ ■ **Security note**: SUID binaries are a classic privilege-escalation target. As a sysadmin, periodically auditing for unexpected SUID binaries is a real security task (we'll do this in Module 10).

1.12 Hidden files (dotfiles)

Any file/directory whose name starts with `.` is "hidden" from a normal `ls` — but not in any security sense, just convention (mostly for config files so they don't clutter your home directory listing).

```
ls -a ~          # see them all
```

You'll commonly encounter: `.bashrc`, `.bash_history`, `.ssh/`, `.gitconfig`, `.profile`, `.config/`. We'll cover the shell-related ones in Module 02.

1.13 `find` — searching the filesystem (a core DevOps tool)

`find` walks a directory tree and filters by almost any criteria you can imagine.

```
find ~/playground -name "*.txt"          # by name (case-sensitive)
find ~/playground -iname "*.TXT"          # case-insensitive
```

```
find ~/playground -type f                 # only regular files
find ~/playground -type d                  # only directories
find ~/playground -type l                   # only symlinks

find / -size +100M 2>/dev/null              # files larger than 100MB (suppress permission errors)
find / -mtime -1 2>/dev/null                 # modified in the last 1 day
find / -mtime +30 2>/dev/null                 # modified more than 30 days ago
find / -mmin -60 2>/dev/null                   # modified in the last 60 minutes

find / -user alex 2>/dev/null                    # owned by user "alex"
find / -perm -002 -type f 2>/dev/null             # world-writable files (security audit!)
find / -perm -4000 -type f 2>/dev/null             # SUID binaries (security audit!)

find ~/playground -empty                             # empty files/directories
```

Acting on results: `-exec` and `-delete`

```
# Find all .log files older than 7 days and delete them
find /var/log/myapp -name "*.log" -mtime +7 -delete

# Find all .sh files and make them executable
find ~/playground -name "*.sh" -exec chmod +x {} \;

# {} is replaced by each matched filename. \; ends the -exec command.
# + instead of \; runs the command once with ALL matches batched together (more efficient):
find ~/playground -name "*.sh" -exec chmod +x {} +
```

■ **DevOps relevance**: "Delete log files older than 14 days," "find every config file referencing a deprecated setting," "find world-writable files for a security audit," "find every file owned by a user we're about to delete" — these are all `find` one-liners, and they show up constantly in cron jobs and scripts.

1.14 Locating commands: `which` , `whereis` , `type` , `locate`

```
which python3      # shows the full path of the binary that would run: /usr/bin/python3
whereis python3      # shows binary, source, and man page locations
type cd                # tells you if something is a builtin, alias, function, or binary
type ls                  # ls: aliased to ls --color=auto  (on Ubuntu, by default!)

# locate uses a prebuilt database (faster than find, but can be stale)
sudo apt install mlocate -y
sudo updatedb            # rebuild the database
locate hostname            # instant search across the whole filesystem
```

Practice exercises

1. Navigate to `/var/log` , then use `cd -` twice and predict where you'll end up before running it.
2. Create a directory structure `~/playground/site/{html,css,js}` in one command using brace expansion + `mkdir -p` .
3. Create a file, make a hard link and a symlink to it. Delete the original. What happens when you `cat` each link now? Explain why.
4. Find every file in `/etc` larger than 10KB, modified in the last 30 days.
5. Create a script file `hello.sh` with `echo "hello"` inside. Try running `./hello.sh` — it'll fail. Fix it using what you learned about permissions, then run it again.
6. Run `find / -perm -4000 -type f 2>/dev/null` on your system and look up what 2-3 of the results actually do.

Next: `02-shell-bash-environment.md`